

1.Explain different types of arrays in php

1 Indexed (Numeric) Arrays

- These arrays use **numeric keys** (0, 1, 2, ...).
- If keys are not specified, PHP automatically assigns them starting from 0.
- Similar to traditional arrays in C/Java.

Creation:

```
$list = array(17, 24, 45, 91);
```

Here, keys will be:

```
0 => 17
```

```
1 => 24
```

```
2 => 45
```

```
3 => 91
```

Accessing elements:

```
echo $list[2]; // Outputs 45
```

You can also create it dynamically:

```
$list[] = 42; // Automatically assigns next numeric key
```

2 Associative Arrays

- These arrays use **string keys** instead of numeric indexes.
- Similar to hash tables or dictionaries in other languages.
- Very useful for storing related data (like name → value).

Creation:

```
$ages = array(  
    "Joe" => 42,  
    "Mary" => 41,  
    "Bif" => 17  
);
```

Accessing elements:

```
echo $ages["Mary"]; // Outputs 41
```

These arrays are commonly used when keys represent meaningful labels.

3 Mixed Arrays

- PHP allows arrays to contain **both numeric and string keys**.
- This is a special feature of PHP.

Example:

```
$stuff = array(  
    "make" => "Cessna",  
    "model" => "C210",  
    "year" => 1960,  
    3 => "sold"  
);
```

Here:

- "make", "model", "year" → string keys
- 3 → numeric key

PHP arrays do not need to be purely indexed or purely associative .

♦ Important Points from Notes

- Arrays are internally stored using **key–value pairs**.
- Elements are stored using a **hashing function** for string keys.
- Arrays maintain the **order of insertion**.
- Arrays can be created:
 1. By assigning values to subscripts.
 2. Using the **array()** construct .

2.Create a cover page that shows time table

Here is a simple and clean Cover Page with Time Table (HTML + CSS).

Cover Page with Time Table

```
<!DOCTYPE html>
<html>
<head>
  <title>Time Table Cover Page</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      background-color: #f2f2f2;
    }

    .cover {
      border: 3px solid black;
      width: 70%;
      margin: 30px auto;
      padding: 30px;
      background-color: white;
    }

    h1 {
      margin-bottom: 5px;
    }

    h2 {
      margin-top: 0;
    }

    table {
      margin: 30px auto;
      border-collapse: collapse;
      width: 90%;
    }

    th, td {
      border: 1px solid black;
      padding: 10px;
    }
```

```

    th {
        background-color: #d9d9d9;
    }
</style>
</head>
<body>

<div class="cover">
    <h1>NMAMIT NITTE</h1>
    <h2>Master of Computer Applications (MCA)</h2>
    <h3>Class Time Table</h3>
    <p><b>Semester:</b> I</p>
    <p><b>Academic Year:</b> 2025-2026</p>

    <table>
        <tr>
            <th>Day</th>
            <th>9:00-10:00</th>
            <th>10:00-11:00</th>
            <th>11:00-12:00</th>
            <th>1:00-2:00</th>
            <th>2:00-3:00</th>
        </tr>
        <tr>
            <td>Monday</td>
            <td>AWT</td>
            <td>DBMS</td>
            <td>Data Mining</td>
            <td>Lab</td>
            <td>Lab</td>
        </tr>
        <tr>
            <td>Tuesday</td>
            <td>PHP</td>
            <td>OS</td>
            <td>AWT</td>
            <td>Seminar</td>
            <td>Library</td>
        </tr>
        <tr>
            <td>Wednesday</td>
            <td>DBMS</td>
            <td>Data Mining</td>
            <td>PHP</td>

```

```

        <td>Lab</td>
        <td>Lab</td>
    </tr>
    <tr>
        <td>Thursday</td>
        <td>OS</td>
        <td>AWT</td>
        <td>DBMS</td>
        <td>Project</td>
        <td>Project</td>
    </tr>
    <tr>
        <td>Friday</td>
        <td>Data Mining</td>
        <td>PHP</td>
        <td>OS</td>
        <td>Seminar</td>
        <td>Sports</td>
    </tr>
</table>
</div>

</body>
</html>

```

Output:

NMAMIT NITTE Master of Computer Applications (MCA) Class Time Table Semester: I Academic Year: 2025-2026					
Day	9:00-10:00	10:00-11:00	11:00-12:00	1:00-2:00	2:00-3:00
Monday	AWT	DBMS	Data Mining	Lab	Lab
Tuesday	PHP	OS	AWT	Seminar	Library
Wednesday	DBMS	Data Mining	PHP	Lab	Lab
Thursday	OS	AWT	DBMS	Project	Project
Friday	Data Mining	PHP	OS	Seminar	Sports

3. Define responsive web design

Responsive Web Design (RWD)

Responsive Web Design is a web design approach that makes a website automatically adjust its layout, images, and content according to the screen size, resolution, and device being used (mobile, tablet, laptop, desktop).

It ensures that the website provides an optimal viewing experience with:

- Easy reading
 - Proper resizing of content
 - No horizontal scrolling
 - Good usability across devices
-

♦ Key Features of Responsive Web Design

1 Flexible Layouts (Fluid Grids)

- Uses relative units like %, **em**, **rem**, **vw**, instead of fixed pixels.
- Layout adjusts proportionally.

2 Flexible Images

- Images scale within their container.
- Example:

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

3 Media Queries

- CSS feature that applies styles based on device screen size.
- Example:

```
@media (max-width: 768px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

♦ Why Responsive Design is Important

- Mobile users are increasing.
 - Improves user experience.
 - Reduces need for separate mobile websites.
 - Better SEO performance.
 - Easier maintenance (one website for all devices).
-

📌 Simple Definition (2–3 Marks Exam Version)

Responsive Web Design is a technique in which a website is designed to automatically adjust its layout and content according to different screen sizes and devices using flexible grids, flexible images, and media queries.

4.how does view port help in building responsive webdesign.

How Viewport Helps in Building Responsive Web Design

♦ What is Viewport?

The **viewport** is the visible area of a web page on a device screen. On mobile devices, browsers use a **virtual viewport** by default, which makes websites appear zoomed out.

♦ Role of Viewport in Responsive Design

Without setting the viewport properly:

- Mobile browsers assume a desktop width (usually around 980px).
- The page shrinks to fit the screen.
- Text becomes very small.
- Layout does not respond correctly.

To fix this, we use the **meta viewport tag**.

♦ Viewport Meta Tag

`<meta name="viewport" content="width=device-width, initial-scale=1.0">`

♦ Meaning of Attributes

- **width=device-width**
→ Sets the page width equal to the device screen width.
 - **initial-scale=1.0**
→ Sets the initial zoom level to 100%.
-

♦ How It Helps in Responsive Design

- 1 Enables proper scaling on mobile devices
 - 2 Makes CSS media queries work correctly
 - 3 Prevents unwanted zooming
 - 4 Ensures layout adjusts according to device width
 - 5 Improves readability and user experience
-

♦ Example Without Viewport

- Website looks zoomed out on mobile.
- User must manually zoom in.

♦ Example With Viewport

- Layout fits perfectly to screen width.
 - Text and images resize properly.
 - Media queries trigger correctly.
-

Short Exam Answer (4–5 Marks)

The viewport helps in responsive web design by defining how a webpage should be displayed on different devices. The meta viewport tag sets the page width equal to the device width and controls scaling. This ensures that layouts, images, and media queries work properly on mobile devices, providing a better user experience.

5.Explain different types of css selectors

1 Simple (Element) Selector

- Selects all elements of a given type.
- Uses the element name.

Syntax:

```
h1 { color: blue; }
```

👉 Applies to all `<h1>` elements.

You can also group selectors:

```
h2, h3 { color: red; }
```

2 Class Selector

- Used to apply style to selected elements with a specific class.
- Defined using a period (.).

Definition:

```
p.normal { color: black; }  
p.warning { color: red; }
```

Usage in HTML:

```
<p class="warning">This is important</p>
```

✓ Allows different styles for the same tag.

3 Generic (Universal Class) Selector

- Applies a class style to multiple types of elements.
- Defined without specifying a tag name.

Example:

```
.sale { color: green; }
```

Used as:

```
<h3 class="sale">Weekend Sale</h3>
<p class="sale">Huge discount available</p>
```

✓ Works for any element with that class.

4 ID Selector

- Applies style to one specific element.
- Uses # symbol.

Syntax:

```
#section14 { color: blue; }
```

HTML:

```
<h2 id="section14">Calico Cats</h2>
```

✓ IDs must be unique in a document.

5 Contextual Selectors

Used when style should apply only to elements in specific positions.

(a) Descendant Selector

```
ul ol { color: red; }
```

✓ Applies to `<ol>` inside `<ul>`.

(b) Child Selector

```
ol > li { color: blue; }
```

✓ Applies only to `<li>` that are direct children of `<ol>`.

6 Pseudo-Class Selectors

- Apply styles based on state or user interaction.

Examples:

```
a:link { color: blue; }  
a:visited { color: purple; }  
a:hover { color: red; }  
h2:hover { color: green; }
```

- ✓ Changes style when user hovers or clicks.
-

Universal Selector

- Applies style to all elements.
- Uses `*`.

```
* { margin: 0; padding: 0; }
```

- ✓ Affects every element in the document.
-

Short Exam Conclusion

The main types of CSS selectors are:

1. Simple (Element) Selector
2. Class Selector
3. Generic Selector
4. ID Selector
5. Contextual Selector (Descendant & Child)
6. Pseudo-class Selector
7. Universal Selector

Selectors allow precise control over how styles are applied to HTML elements.

6.Explain bootstrap with system and how it works

Bootstrap – Definition

Bootstrap is a free, open-source front-end framework used to design responsive and mobile-first websites quickly.

It includes pre-written **CSS, JavaScript, and components** for layout, forms, buttons, navigation bars, modals, and more.

It was originally developed by Twitter developers and is now one of the most widely used UI frameworks.

Bootstrap System (Grid System)

The core of Bootstrap is its **Grid System**, which helps create responsive layouts.

♦ 1. Container

Bootstrap layouts start with a container.

```
<div class="container">  
</div>
```

- **.container** → Fixed width layout
 - **.container-fluid** → Full width layout
-

♦ 2. Rows

Rows are used to group columns horizontally.

```
<div class="row">  
</div>
```

♦ 3. Columns

Bootstrap divides the screen into **12 equal columns**.

Example:

```
<div class="row">  
  <div class="col-6">Column 1</div>  
  <div class="col-6">Column 2</div>
```

</div>

Here:

- `col-6` → Takes 6 out of 12 columns (50% width)
 - $6 + 6 = 12$ (full row)
-

♦ Responsive Breakpoints

Bootstrap uses different classes for different screen sizes:

- `col-sm` → Small devices
- `col-md` → Medium devices
- `col-lg` → Large devices
- `col-xl` → Extra large devices

Example:

```
<div class="col-md-4 col-sm-12">
```

- On medium screens → 4 columns
 - On small screens → Full width (12 columns)
-

How Bootstrap Works

Step 1 Include Bootstrap

Add Bootstrap CSS and JS via CDN:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
```

Step 2 Use Bootstrap Classes

Bootstrap works using **predefined classes**.

Example – Button:

```
<button class="btn btn-primary">Click Me</button>
```

Here:

- **btn** → Base button style
 - **btn-primary** → Blue colored button
-

Step 3 Responsive Layout

Bootstrap uses:

- Grid system (12-column layout)
- Media queries
- Flexible containers

So layout automatically adjusts based on screen size.

Key Features of Bootstrap

1. Responsive Grid System
 2. Pre-styled Components (Navbar, Cards, Alerts, Modals)
 3. Built-in Icons
 4. JavaScript Plugins
 5. Mobile-first design
 6. Customizable themes
-

Example of Simple Bootstrap Layout

```
<div class="container">  
  <div class="row">  
    <div class="col-md-4 bg-primary text-white">Column 1</div>  
    <div class="col-md-4 bg-success text-white">Column 2</div>  
    <div class="col-md-4 bg-danger text-white">Column 3</div>  
  </div>  
</div>
```

On large screens → 3 columns
On small screens → stacked vertically

Short Exam Answer (5–6 Marks)

Bootstrap is a front-end framework used to develop responsive and mobile-first websites. It provides a 12-column grid system, pre-designed components, CSS utilities, and JavaScript plugins. It works by using predefined CSS classes and media queries to automatically adjust layouts according to different screen sizes.

7.Semantic elements of HTML5 (explain).

Semantic Elements of HTML5

◆ Definition

Semantic elements in HTML5 are tags that clearly describe the meaning and purpose of the content they contain.

They make the structure of a webpage more meaningful for:

- Browsers
- Developers
- Search engines
- Screen readers

Unlike `<div>` and `<span>`, semantic elements tell **what the content represents**, not just how it looks.

Common Semantic Elements in HTML5

1 `<header>`

- Represents introductory content.
- Usually contains logo, title, navigation, or headings.
- Can appear at the top of a page or inside a section/article.

```
<header>  
  <h1>My Website</h1>  
</header>
```

2 <nav>

- Defines navigation links.
- Used for menus, sidebar links, table of contents.

```
<nav>
  <a href="#">Home</a>
  <a href="#">About</a>
</nav>
```

3 <section>

- Defines a thematic grouping of content.
- Usually has a heading.
- Used to divide content into logical parts.

```
<section>
  <h2>Introduction</h2>
  <p>Content goes here...</p>
</section>
```

4 <article>

- Represents independent, self-contained content.
- Can be reused or distributed independently.
- Examples: blog post, news article, forum post.

```
<article>
  <h2>News Title</h2>
  <p>News content...</p>
</article>
```

5 <aside>

- Represents content related to the main content.
- Used for sidebars, advertisements, related links.


```
<aside>
  <p>Related Articles</p>
</aside>
```

6 <footer>

- Represents footer of a page or section.
- Contains author info, copyright, links, contact info.

```
<footer>
  <p>© 2026 My Website</p>
</footer>
```

7 <main>

- Specifies the main content of the document.
- Should not contain repeated content like navigation or footer.

```
<main>
  <h2>Main Content</h2>
</main>
```

8 <figure> and <figcaption>

- `<figure>` groups media content.
- `<figcaption>` provides caption.

```
<figure>
  
  <figcaption>Sample Image</figcaption>
</figure>
```

Advantages of Semantic Elements

1. Improves readability of code
2. Better SEO (Search Engine Optimization)

3. Enhances accessibility
 4. Easier maintenance
 5. Clearly defines webpage structure
-

Short Exam Answer (5 Marks)

Semantic elements in HTML5 are tags that describe the meaning of content. Examples include `<header>`, `<nav>`, `<section>`, `<article>`, `<aside>`, `<footer>`, and `<main>`. These elements improve code clarity, SEO, accessibility, and document structure by clearly defining different parts of a webpage.

8.Explain MIME type specification in response or request transaction

MIME Type Specification in HTTP Request–Response Transaction

♦ What is MIME?

MIME (Multipurpose Internet Mail Extensions) is used to specify the **type and format of data** being transmitted between a client (browser) and a server.

In Web communication, MIME types tell the browser **how to interpret and display the received data**.

MIME format:

type/subtype

Examples:

- `text/html`
 - `text/plain`
 - `image/jpeg`
 - `image/gif`
 - `video/mpeg`
-

MIME in HTTP Transaction

HTTP communication has two phases:

1. **Request**
2. **Response**

Each message contains:

- Header
- Body

MIME type information appears in the **header fields**.

1] MIME in HTTP Request

In a request, the browser tells the server what type of content it can accept using the **Accept header field**.

Example:

```
GET /index.html HTTP/1.1
Host: example.com
Accept: text/html
Accept: image/gif
Accept: text/*
```

♦ **Purpose:**

- Specifies preferred MIME types.
 - Helps server choose appropriate response format.
 - If multiple formats are available, server selects best match.
-

2] MIME in HTTP Response

In the response, the server specifies the format of the returned data using the **Content-Type header field**.

Example:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 364
```

♦ Purpose:

- Tells browser how to process the body.
 - Browser selects correct rendering method.
 - For example:
 - `text/html` → Display as web page
 - `image/jpeg` → Display as image
 - `application/pdf` → Open PDF viewer
-

📌 Structure of MIME Specification

MIME types follow:

type/subtype

Common Types:

Type	Subtype Examples
text	plain, html
image	gif, jpeg
video	mpeg, quicktime

Experimental subtypes begin with:

x-

Example:

video/x-msvideo

📌 Importance of MIME in HTTP

1. Enables correct rendering of documents.
2. Ensures browser uses proper software for display.

3. Supports different content types (text, image, audio, video).
 4. Essential for file downloads and media streaming.
-

Short Exam Answer (5–6 Marks)

MIME type specification is used in HTTP transactions to define the format of data being transferred. In the request phase, the browser uses the Accept header to specify preferred MIME types. In the response phase, the server uses the Content-Type header to indicate the format of the returned document. MIME types follow the format type/subtype, such as text/html or image/jpeg. This helps the browser correctly interpret and display the content.

9.Explain the role of HTML5 elements.

i. nav

ii. aside

iii. Section

Role of HTML5 Elements

HTML5 introduced semantic elements to clearly define the structure and meaning of webpage content. The elements `<nav>`, `<aside>`, and `<section>` help organize content logically and improve readability, SEO, and accessibility.

i) `<nav>` Element

◆ Role:

The `<nav>` element is used to define a section that contains **navigation links**.

◆ Purpose:

- Provides links to other pages or parts of the same page.
- Used for menus, navigation bars, table of contents.
- Helps search engines identify main navigation links.

◆ Example:

```
<nav>  
<a href="#">Home</a>
```

```
<a href="#">About</a>
<a href="#">Contact</a>
</nav>
```

♦ Importance:

- Improves website structure.
 - Enhances accessibility for screen readers.
 - Helps search engines understand page navigation.
-

ii) <aside> Element

♦ Role:

The `<aside>` element represents content that is **related to the main content** but not part of it.

♦ Purpose:

- Used for sidebars.
- Displays advertisements.
- Shows related articles or additional information.

♦ Example:

```
<aside>
<h3>Related Links</h3>
<p>More information about this topic.</p>
</aside>
```

♦ Importance:

- Separates secondary content from main content.
 - Improves page layout organization.
 - Enhances semantic clarity.
-

iii) <section> Element

♦ Role:

The `<section>` element defines a **thematic grouping of content** within a document.

♦ **Purpose:**

- Divides content into logical parts.
- Usually contains a heading.
- Used for chapters, topics, or different sections of a page.

♦ **Example:**

```
<section>
<h2>Introduction</h2>
<p>This section explains the basics.</p>
</section>
```

♦ **Importance:**

- Organizes content clearly.
 - Improves SEO.
 - Makes document structure meaningful.
-

Short Exam Conclusion

- `<nav>` defines navigation links.
- `<aside>` represents related or side content.
- `<section>` defines thematic grouping of content.

These HTML5 elements improve webpage structure, accessibility, and search engine optimization.

10.Explain font-style, font-variant, font-weight properties of CSS

CSS Font Properties

CSS provides several properties to control the appearance of text. Among them, **font-style**, **font-variant**, and **font-weight** are used to modify the look of fonts.

font-style

♦ Purpose:

The **font-style** property specifies whether the text should be displayed in a normal, italic, or oblique style.

♦ Syntax:

```
selector {  
  font-style: value;  
}
```

♦ Values:

- **normal** → Default text style
- **italic** → Italic text
- **oblique** → Slanted version of normal text

♦ Example:

```
p {  
  font-style: italic;  
}
```

👉 Makes paragraph text italic.

2 font-variant

♦ Purpose:

The **font-variant** property is mainly used to display text in **small capital letters**.

♦ Syntax:

```
selector {  
  font-variant: value;  
}
```

♦ Values:

- **normal** → Default text
- **small-caps** → Displays lowercase letters as smaller uppercase letters

♦ Example:

```
h2 {  
  font-variant: small-caps;  
}
```

👉 Displays heading in small capital letters.

3 font-weight

♦ Purpose:

The **font-weight** property specifies the **thickness (boldness)** of text.

♦ Syntax:

```
selector {  
  font-weight: value;  
}
```

♦ Values:

- **normal** → Normal thickness
- **bold** → Bold text
- **bolder** → Heavier than parent
- **lighter** → Lighter than parent
- Numeric values → 100 to 900

Common numeric values:

- 400 → Normal
- 700 → Bold

♦ Example:

```
strong {  
  font-weight: bold;  
}
```

Or

```
p {  
  font-weight: 700;
```

}

👉 Makes text bold.

📌 Short Exam Answer (5 Marks)

- **font-style** specifies italic or normal text.
- **font-variant** displays text in small capital letters.
- **font-weight** controls the boldness or thickness of text.

These properties help improve the visual presentation of text in a webpage.